

B.Sc. Computer Science

Unit IV

Package

Java में **Package** का मतलब है एक ऐसा "कंटेनर" या "फ़ोल्डर" जिसमें एक जैसी (related) क्लासेस, इंटरफेस और सब-पैकेजेस को रखा जाता है।

इसे एक आसान उदाहरण से समझें: जैसे आपके कंप्यूटर में आप गानों के लिए 'Music' फ़ोल्डर और अपनी फाइल्स के लिए 'Documents' फ़ोल्डर बनाते हैं ताकि सब कुछ व्यवस्थित (organized) रहे, वैसे ही Java में **Packages** का इस्तेमाल होता है।

1. Package के फायदे (Why we use it?)

1. **Code Reusability:** एक पैकेज में लिखी क्लास को दूसरे पैकेज में import करके इस्तेमाल किया जा सकता है।
2. **Name Conflicts:** अगर दो अलग-अलग लोग एक ही नाम की क्लास (जैसे Student.java) बनाते हैं, तो पैकेजेस उन्हें अलग रखने में मदद करते हैं। (जैसे school.Student और college.Student)।
3. **Access Protection:** protected और default मॉडिफायर्स की मदद से हम यह तय कर सकते हैं कि कौन सी क्लास पैकेज के बाहर दिखेगी और कौन सी नहीं।
4. **Easy Maintenance:** हजारों क्लासेज होने पर उन्हें ढूँढना आसान हो जाता है।

2. Types of Packages (पैकेज के प्रकार)

Java में पैकेज दो तरह के होते हैं:

1. **Built-in Packages:** जो Java API के साथ पहले से आते हैं। (जैसे java.util, java.lang, java.io, java.sql)।
2. **User-defined Packages:** जिन्हें हम (डेवलपर) अपनी ज़रूरत के हिसाब से बनाते हैं।

3. Package कैसे बनाते और इस्तेमाल करते हैं?

A. पैकेज बनाना (Create)

पैकेज बनाने के लिए क्लास में सबसे ऊपर package कीवर्ड का उपयोग किया जाता है।

Java

```
// यह फाइल 'mypack' नाम के फोल्डर में होनी चाहिए
package mypack;

public class MyClass {
    public void display() {
        System.out.println("Hello from Package!");
    }
}
```

B. पैकेज को इस्तेमाल करना (Import)

किसी दूसरे पैकेज की क्लास इस्तेमाल करने के लिए import कीवर्ड का उपयोग होता है।

Java

```
import mypack.MyClass; // या import mypack.*; (सब कुछ इस्तेमाल करने के लिए)

public class Test {
    public static void main(String args[]) {
        MyClass obj = new MyClass();
        obj.display();
    }
}
```

4. Package Naming Convention (कॉलेज नोट्स के लिए महत्वपूर्ण)

इंडस्ट्री में पैकेज का नाम रखने का एक खास तरीका है ताकि नाम कभी क्लैश न हों:

- हमेशा **Lowercase** में लिखें।
- कंपनी की वेबसाइट का उल्टा नाम इस्तेमाल करें।
- उदाहरण: अगर वेबसाइट google.com है, तो पैकेज का नाम com.google.projectname होगा।

5. Summary Table for Quick Revision

Feature	Detail
Keyword	package (बनाने के लिए), import (इस्तेमाल के लिए)।
Position	क्लास की सबसे पहली लाइन होनी चाहिए।
Directory	पैकेज का नाम फोल्डर के नाम के बराबर होना चाहिए।
Conflict	एक ही नाम की दो क्लास अलग-अलग पैकेज में रह सकती हैं।

एक ज़रूरी बात: क्या आप जानते हैं कि Java में java.lang इकलौता ऐसा पैकेज है जिसे हमें कभी import नहीं करना पड़ता? (जैसे String और System क्लासेस इसी का हिस्सा हैं)।

Java में सैकड़ों **Built-in Packages** हैं, लेकिन एक डेवलपर के रूप में आपको सबसे ज्यादा इस्तेमाल होने वाले 5-6 पैकेजेस के बारे में पता होना चाहिए। ये पैकेजेस Java Runtime Environment (JRE) के साथ पहले से आते हैं।

यहाँ सबसे महत्वपूर्ण Built-in Packages की जानकारी कॉलेज नोट्स के स्टाइल में दी गई है:

Frequently Used Built-in Packages in Java

1. java.lang (The Default Package)

यह Java का सबसे महत्वपूर्ण पैकेज है। इसकी खासियत यह है कि इसे **Import करने की ज़रूरत नहीं होती**; यह अपने आप हर Java प्रोग्राम में उपलब्ध रहता है।

- **महत्वपूर्ण Classes:** String, System, Math, Object, Thread, Exception आदि।
- **उपयोग:** बेसिक ऑपरेशन्स के लिए (जैसे System.out.println() या Math कैलकुलेशन)।

2. java.util (Utility Package)

इसे "Utility" पैकेज कहा जाता है क्योंकि इसमें डेटा को स्टोर और मैनेज करने के लिए बहुत शक्तिशाली क्लासेस होती हैं।

- **महत्वपूर्ण Classes:** ArrayList, HashMap, Scanner (इनपुट के लिए), Date, Random (रैंडम नंबर के लिए)।
- **उपयोग:** डेटा कलेक्शंस (Data Structures) को हैंडल करने और यूजर से इनपुट लेने के लिए।

3. java.io (Input/Output Package)

यह पैकेज फाइल्स के साथ काम करने और डेटा को स्ट्रीम (Stream) करने के लिए इस्तेमाल होता है।

- **महत्वपूर्ण Classes:** File, FileInputStream, FileOutputStream, BufferedReader, PrintWriter।
- **उपयोग:** कंप्यूटर की फाइल्स को पढ़ने (Read) और लिखने (Write) के लिए।

4. java.net (Networking Package)

अगर आप ऐसा एप्लीकेशन बना रहे हैं जो इंटरनेट या नेटवर्क के ज़रिए दूसरे कंप्यूटर से बात करे, तो यह पैकेज काम आता है।

- **महत्वपूर्ण Classes:** Socket, ServerSocket, URL, HttpURLConnection।
- **उपयोग:** क्लाइंट-सर्वर प्रोग्रामिंग और URL हैंडलिंग के लिए।

5. java.awt & javax.swing (GUI Packages)

इनका उपयोग डेस्कटॉप एप्लीकेशन (Windows Apps) बनाने के लिए किया जाता है जहाँ बटन्स, विंडोज़ और टेक्स्ट बॉक्स होते हैं।

- **महत्वपूर्ण Classes:** JButton, JFrame, JTextField, Color, Font।
- **उपयोग:** Graphical User Interface (GUI) बनाने के लिए।

6. java.sql (Database Package)

जब आपको Java प्रोग्राम को किसी डेटाबेस (जैसे MySQL या Oracle) से कनेक्ट करना हो, तब इसका इस्तेमाल होता है।

- **महत्वपूर्ण Classes:** Connection, Statement, ResultSet, DriverManager।
- **उपयोग:** डेटाबेस में डेटा सेव करने या वहाँ से डेटा निकालने के लिए।

Summary Table for Quick Revision

Package Name	Main Purpose	Auto-Imported?
java.lang	Basic language support (String, Math)	Yes
java.util	Collections and Scanner	No
java.io	File Handling	No
java.net	Networking & Sockets	No
java.sql	Database Connectivity (JDBC)	No
javax.swing	GUI (Desktop Apps)	No

इसे इस्तेमाल करने का सही तरीका (Standard Practice):

हमेशा अपनी क्लास के सबसे ऊपर इन्हें import करें:

Java

```
import java.util.Scanner; // सिर्फ Scanner के लिए
```

```
import java.util.*; // util के अंदर की सभी क्लासेज के लिए
```

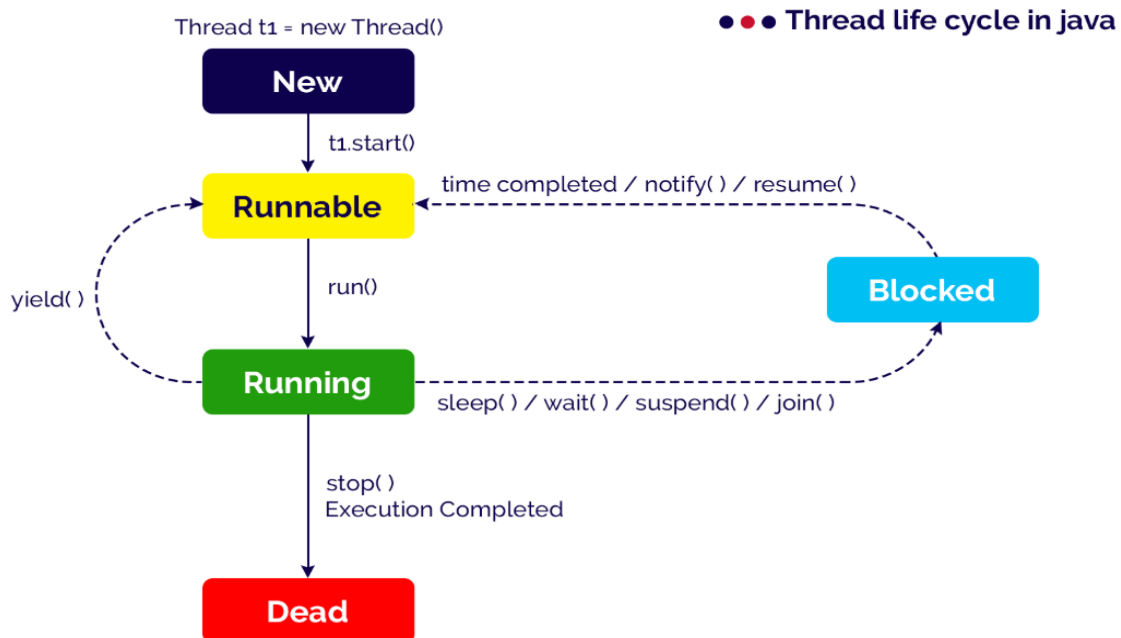
Thread

एक थ्रेड (Thread) किसी **प्रोसेस (Process)** के भीतर निष्पादन (Execution) की सबसे छोटी इकाई होती है। इसे "**लाइटवेट सब-प्रोसेस**" भी कहा जाता है क्योंकि यह एक अलग रास्ता होता है जिससे प्रोग्राम का कोड रन होता है।

Definition (परिभाषा): Java में **Thread** एक प्रोग्राम के अंदर का सबसे छोटा हिस्सा (Smallest unit) होता है जो स्वतंत्र रूप से चल सकता है। एक भारी प्रोग्राम को छोटे-छोटे टुकड़ों में बाँटना और उन्हें एक साथ चलाना ही **Multithreading** कहलाता है।

सरल शब्दों में: जैसे एक वेब ब्राउज़र में एक साथ गाना बजना, फाइल डाउनलोड होना और टाइपिंग करना, ये सब अलग-अलग थ्रेड्स का काम है।

Thread Life Cycle (The 5 States)



Java में थ्रेड की अवस्थाएं Thread.State enum द्वारा परिभाषित की जाती हैं:

1. **New (Born):** Thread t = new Thread(); - यहाँ ऑब्जेक्ट बन गया है लेकिन एक्सीक्यूशन शुरू नहीं हुआ है।
2. **Runnable (Ready):** t.start(); - थ्रेड "Thread Scheduler" के पास चला गया है। यह चलने के लिए तैयार है।
3. **Running:** जब Scheduler थ्रेड को CPU टाइम देता है, तो वह Running स्टेट में आता है और run() मेथड एक्सीक्यूट होता है।
4. **Timed Waiting / Blocked:** जब थ्रेड sleep(ms), wait(), या किसी resource (जैसे I/O) का इंतज़ार कर रहा हो।
5. **Terminated (Dead):** जब run() मेथड पूरी तरह समाप्त हो जाता है या कोई अनहैंडल्ड एरर आ जाती है।

3. Creating Threads (Syntaxes)

Method 1: Extending the Thread Class

इसका उपयोग तब करें जब आप Thread क्लास के व्यवहार को बदलना चाहते हैं।

```
class MyProcessor extends Thread {
    public void run() {
        System.out.println("Thread ID: " + Thread.currentThread().getId() + " is running.");
    }
}
```

Method 2: Implementing the Runnable Interface

यह सबसे अच्छा तरीका है (Best Practice) क्योंकि यह आपको किसी दूसरी क्लास को भी extend करने की अनुमति देता है।

```
class MyTask implements Runnable {
    public void run() {
        System.out.println("Task execution started.");
    }
}
```

4. Using and Controlling Threads

Action	Method Name	Description
Start	start()	थ्रेड को रजिस्टर करता है और run() को शुरू करता है।
Pause	Thread.sleep(ms)	थ्रेड को कुछ मिलीसेकंड के लिए रोकता है।
Join	join()	करंट थ्रेड को तब तक रोकता है जब तक कि दूसरा थ्रेड खत्म न हो जाए।
Interrupt	interrupt()	थ्रेड को संकेत देता है कि उसे अपना काम बंद कर देना चाहिए।

5. Thread Exceptions and Stopping

A. Thread Exceptions

सबसे प्रमुख एक्सेप्शन **InterruptedException** है। यह एक **Checked Exception** है।

- **Reason:** जब कोई थ्रेड सोया हुआ (sleep) हो और हम उसे interrupt() कर दें, तो यह एरर आता है।

B. Stopping a Thread

Java में stop() मेथड **Deprecated** (असुरक्षित) है क्योंकि यह डेटा को कॉरुप्ट कर सकता है। थ्रेड को रोकने के लिए हमेशा एक "Interrupt" या "Volatile Boolean" का उपयोग करें।

6. Comprehensive Code Example

```
class Worker implements Runnable {
    // Volatile ensures visibility across different threads
    private volatile boolean exit = false;

    public void stopWorker() {
        exit = true;
    }

    @Override
```

```

    public void run() {
        System.out.println("Worker thread started...");
        while (!exit) {
            try {
                System.out.println("Worker is performing background tasks...");
                Thread.sleep(1000); // Wait for 1 second
            } catch (InterruptedException e) {
                System.out.println("Thread was interrupted! Cleaning up...");
                break; // Exit the loop if interrupted
            }
        }
        System.out.println("Worker thread terminated safely.");
    }
}

public class ThreadManagement {
    public static void main(String[] args) {
        Worker workerLogic = new Worker();
        Thread thread = new Thread(workerLogic);

        thread.start(); // Using thread

        try {
            Thread.sleep(3000); // Main thread waits 3 seconds
        } catch (InterruptedException e) {}

        System.out.println("Main: Stopping worker now...");
        workerLogic.stopWorker(); // Safely stopping thread
    }
}

```

Key Takeaways (महत्वपूर्ण बातें)

- **Main Thread:** हर जावा प्रोग्राम कम से कम एक थ्रेड (main) पर चलता है।
- **Daemon Threads:** ये बैकग्राउंड थ्रेड होते हैं (जैसे Garbage Collector) जो प्रोग्राम के खत्म होते ही अपने आप बंद हो जाते हैं।
- **Concurrency vs Parallelism:** एक CPU कोर पर थ्रेड्स के बीच स्विच करना Concurrency है, जबकि मल्टी-कोर CPU पर एक साथ थ्रेड्स चलाना Parallelism है।

Next Step: क्या आप **Thread Synchronization** (ताकि दो थ्रेड्स एक साथ एक ही वेरिएबल को खराब न करें) सीखना चाहेंगे या फिर **Exception Handling** के throw और throws की बारीकियां देखें?

1. What are Errors?

प्रोग्रामिंग की दुनिया में, **Error** एक ऐसी स्थिति है जब कोई प्रोग्राम उम्मीद के मुताबिक काम नहीं करता है। यह कोडिंग के दौरान की गई गलती, गलत लॉजिक या सिस्टम संसाधनों की कमी के कारण हो सकता है।

सामान्यतः एरर्स को तीन मुख्य श्रेणियों में बांटा जाता है:

- 1. **Compile-Time (Syntax) Errors:** जब कोड लिखने के नियमों (व्याकरण) में गलती हो। कंपाइलर इसे पहले ही पकड़ लेता है।
 - *Example:* `int x = 10 (बिना ; के)।`
- 2. **Runtime Errors:** जब प्रोग्राम कंपाइल तो हो जाता है, लेकिन चलने के दौरान क्रैश हो जाता है।
 - *Example:* किसी संख्या को 0 से भाग देना।
- 3. **Logical Errors:** प्रोग्राम चलता है, लेकिन परिणाम गलत आता है।
 - *Example:* `2 + 2` का उत्तर 5 आना (गलत फॉर्मूला)।

Java Errors

- 4. जावा में **"Error"** शब्द का एक विशेष तकनीकी अर्थ है। जावा की `Throwable` क्लास के दो मुख्य हिस्से होते हैं: **Exception** और **Error**।
- 5. **Java Error (`java.lang.Error`) की परिभाषा:** यह उन गंभीर और लाइलाज (Irrecoverable) समस्याओं को दर्शाता है जो प्रोग्राम के नियंत्रण से बाहर होती हैं। ये समस्याएँ आमतौर पर जावा वर्चुअल मशीन (JVM) या हार्डवेयर के स्तर पर होती हैं।
- 6. **मुख्य विशेषता:** आप इन्हें `try-catch` ब्लॉक से ठीक नहीं कर सकते। यदि जावा एरर आता है, तो प्रोग्राम का बंद होना तय है।

Types of Java Errors (जावा एरर के प्रकार)

- 7. जावा में मुख्य रूप से निम्नलिखित प्रकार के एरर पाए जाते हैं:

Error Type	Description
VirtualMachineError	जब JVM खराब हो जाए या उसे काम जारी रखने के लिए संसाधन न मिलें।
OutOfMemoryError	जब प्रोग्राम इतने ऑब्जेक्ट बना दे कि RAM (Heap Space) भर जाए।
StackOverflowError	जब कोई मेथड खुद को इतनी बार कॉल करे (Infinite Recursion) कि मेमोरी स्टैक भर जाए।
NoClassDefFoundError	जब कंपाइलर की गई क्लास फाइल रनटाइम के दौरान गायब हो जाए।

Java Exception (The `Exception` Class)

जावा में, एक **Exception** वह घटना (Event) है जो प्रोग्राम के चलने (**Execution**) के दौरान होती है और निर्देशों के सामान्य प्रवाह (Normal Flow) को बाधित करती है। तकनीकी रूप से, Exception एक **Class** है जो `java.lang.Exception` से आती है।

1. Definition (परिभाषा)

जब आपके कोड में कोई गड़बड़ी होती है (जैसे गलत इनपुट या फाइल का न मिलना), तो JVM उस विशिष्ट समस्या से संबंधित **Exception Class** का एक **Object** बनाता है। इस ऑब्जेक्ट में एरर का प्रकार और प्रोग्राम की स्थिति की जानकारी होती है।

Types of Java Exceptions (एक्सेप्शन के प्रकार)

जावा में एक्सेप्शन्स को मुख्य रूप से दो श्रेणियों में बांटा गया है, जो इस बात पर निर्भर करती हैं कि कंपाइलर उन्हें कब चेक करता है:

A. Checked Exceptions (कंपाइल-टाइम)

ये वे एक्सेप्शन्स हैं जिन्हें जावा कंपाइलर **Compile-time** पर ही चेक करता है। यदि आप इन्हें try-catch या throws से हैंडल नहीं करते, तो आपका प्रोग्राम कंपाइल ही नहीं होगा।

- **Use Case:** ये आमतौर पर बाहरी कारकों (External factors) जैसे फाइल या नेटवर्क से संबंधित होते हैं।

B. Unchecked Exceptions (रनटाइम)

ये वे एक्सेप्शन्स हैं जो **Runtime** पर आते हैं। कंपाइलर इन्हें चेक नहीं करता, इसलिए इन्हें 'Runtime Exceptions' भी कहा जाता है।

- **Use Case:** ये आमतौर पर प्रोग्रामिंग लॉजिक की गलतियों के कारण होते हैं।

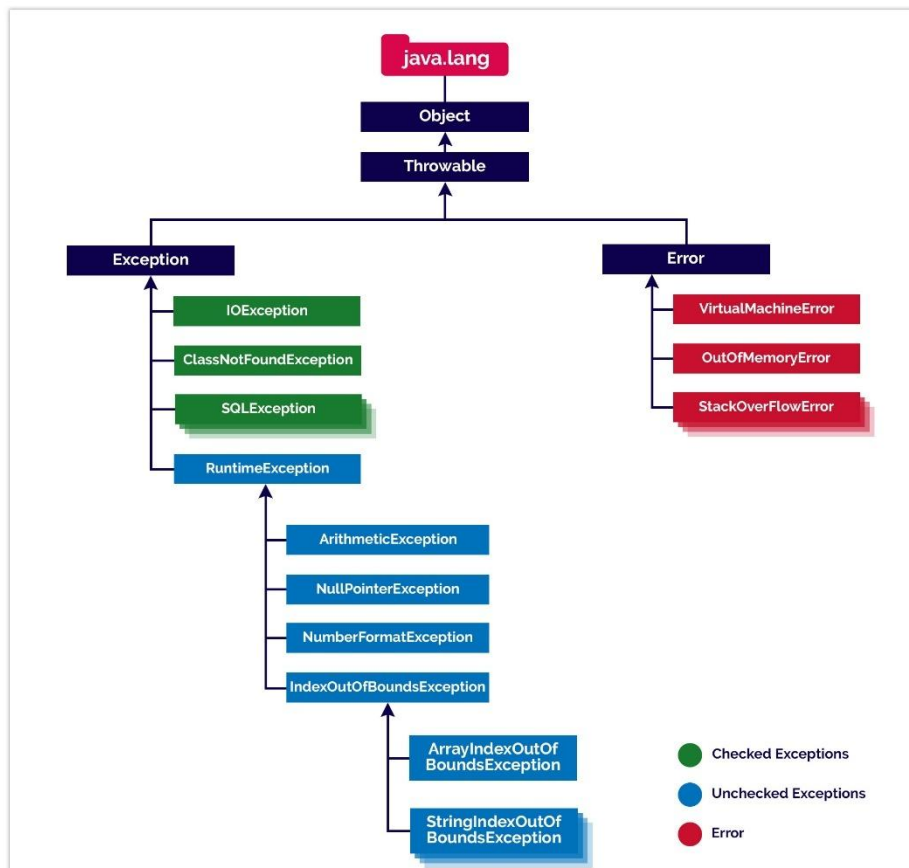
4. Common Exception Classes (प्रमुख उदाहरण)

Exception Type	Description (विवरण)	Category
ArithmeticException	जब गणितीय रूप से कुछ गलत हो (जैसे 10/0)।	Unchecked
NullPointerException	जब आप किसी null ऑब्जेक्ट के मेथड को कॉल करने की कोशिश करें।	Unchecked
ArrayIndexOutOfBoundsException	जब आप ऐरे की लिमिट (Index) से बाहर डेटा एक्सेस करें।	Unchecked
IOException	जब इनपुट-आउटपुट ऑपरेशन (जैसे फाइल पढ़ना) विफल हो जाए।	Checked
FileNotFoundException	जब आप ऐसी फाइल खोलना चाहें जो सिस्टम में मौजूद ही नहीं है।	Checked

Difference Between Java Error and Exception

जावा में Error और Exception दोनों ही Throwable क्लास की संतानें (Sub-classes) हैं, लेकिन इनके काम करने का तरीका और उद्देश्य पूरी तरह अलग है।

आधार (Basis)	Java Error (जावा एरर)	Java Exception (जावा एक्सेप्शन)
1. क्लास पाथ	यह java.lang.Error क्लास का हिस्सा है।	यह java.lang.Exception क्लास का हिस्सा है।
2. मूल कारण (Source)	यह बाहरी वातावरण या JVM की समस्याओं के कारण होता है।	यह प्रोग्रामिंग लॉजिक या गलत यूजर डेटा के कारण होता है।
3. सुधार (Recovery)	यह Irrecoverable (लाइलाज) है। प्रोग्राम को बंद करना ही पड़ता है।	यह Recoverable है। इसे संभालकर प्रोग्राम को जारी रखा जा सकता है।
4. हैंडलिंग (Handling)	इन्हें try-catch ब्लॉक से हैंडल करने की कोशिश नहीं करनी चाहिए।	इन्हें try-catch और finally का उपयोग करके आसानी से हैंडल किया जा सकता है।
5. प्रकार (Types)	ये हमेशा Unchecked होते हैं (रनटाइम पर ही पता चलते हैं)।	ये Checked (कंपाइल-टाइम) और Unchecked दोनों हो सकते हैं।
6. जिम्मेदारी	इसके लिए सिस्टम एडमिन या हार्डवेयर जिम्मेदार होता है।	इसके लिए प्रोग्रामर (यानी आप) जिम्मेदार होते हैं।
7. मुख्य उदाहरण	StackOverflowError, OutOfMemoryError	NullPointerException, ArithmeticException, IOException



Exception Handling (एक्सेप्शन हैंडलिंग)

Definition: यह जावा का एक ऐसा मैकेनिज्म (Mechanism) है जो रनटाइम एरर्स (Runtime Errors) को संभालता है। इसका मुख्य उद्देश्य प्रोग्राम को अचानक बंद (Crash) होने से बचाना और कोड के सामान्य प्रवाह (Normal Flow) को बनाए रखना है।

जब कोई गड़बड़ी होती है, तो JVM एक **Exception Object** बनाता है। हैंडलिंग का मतलब है उस ऑब्जेक्ट को पकड़ना और उसका समाधान करना।

2. Keywords and Their Uses (कीवर्ड्स और उनके उपयोग)

जावा में एक्सेप्शन हैंडलिंग के लिए **5 मुख्य कीवर्ड्स** का उपयोग किया जाता है:

Keyword	Use (उपयोग)
try	इसका उपयोग उस कोड ब्लॉक को घेरने के लिए किया जाता है जिसमें एक्सेप्शन आने की संभावना हो। इसे अकेले इस्तेमाल नहीं कर सकते।
catch	इसका उपयोग try ब्लॉक में आए एक्सेप्शन को पकड़ने और उसे हैंडल करने के लिए किया जाता है। यह try के ठीक बाद आता है।
finally	इसका उपयोग महत्वपूर्ण कोड (जैसे क्लोजिंग फाइल) को चलाने के लिए किया जाता है। यह चाहे एक्सेप्शन आए या न आए, हमेशा चलता है।
throw	इसका उपयोग किसी एक्सेप्शन को मैन्युअली (Explicitly) फेंकने के लिए किया जाता है।
throws	इसका उपयोग मेथड सिग्नेचर में किया जाता है यह बताने के लिए कि यह मेथड भविष्य में कौन-सा एक्सेप्शन दे सकता है।

3. Syntax (लिखने का तरीका)

आपके नोट्स के लिए यह स्टैंडर्ड सिंटैक्स है:

```
try {  
    // 1. Risky code (जहाँ गलती हो सकती है)  
}  
catch (ExceptionClassName e) {  
    // 2. Handling code (गलती होने पर क्या करना है)  
}  
finally {  
    // 3. Cleanup code (जो हमेशा चलेगा)  
}
```

4. Full Example (एक साथ सभी का उपयोग)

यहाँ एक प्रैक्टिकल उदाहरण है जो try, catch, और finally को एक साथ समझाता है:

Java

```

public class ExceptionDemo {
    public static void main(String[] args) {
        System.out.println("Program Started...");

        try {
            int a = 10;
            int b = 0;
            int result = a / b; // यहाँ ArithmeticException आएगा
            System.out.println("Result: " + result);
        }
        catch (ArithmeticException e) {
            // JVM इस ब्लॉक में आता है क्योंकि b = 0 है
            System.out.println("Error: Zero division is not allowed in Math.");
        }
        finally {
            // यह हमेशा प्रिंट होगा
            System.out.println("Cleaning up resources... Execution completed.");
        }

        System.out.println("Program finished safely.");
    }
}

```

Finally (हमेशा चलने वाला ब्लॉक)

उपयोग: इसका उपयोग क्लीनअप (Cleanup) के लिए किया जाता है। चाहे try में एरर आए या न आए, finally के अंदर का कोड हमेशा चलेगा।

Simple Example:

Java

```

public class FinallyDemo {
    public static void main(String[] args) {
        try {
            int a = 10 / 2; // कोई एरर नहीं है
            System.out.println("Result: " + a);
        } catch (Exception e) {
            System.out.println("Error caught.");
        } finally {
            System.out.println("Finally: I will always run!");
        }
    }
}

```

2. Throw

उपयोग: इसका उपयोग किसी विशिष्ट एक्सेप्शन को **मैन्युअली (Explicitly)** फेंकने के लिए किया जाता है। यह अक्सर if-else के साथ लॉजिक चेक करने के लिए इस्तेमाल होता है।

Simple Example:

Java

```
public class ThrowDemo {
    static void checkAge(int age) {
        if (age < 18) {
            // खुद से एक्सेप्शन पैदा करना
            throw new ArithmeticException("Aakash, you are not eligible to vote.");
        } else {
            System.out.println("Access granted!");
        }
    }

    public static void main(String[] args) {
        checkAge(15); // यह लाइन एरर थ्रो करेगी
    }
}
```

3. Throws (चेतावनी देना)

उपयोग: इसका उपयोग मेथड के सिग्नेचर में किया जाता है। यह कंपाइलर को बताता है कि "इस मेथड के अंदर कोई एक्सेप्शन आ सकता है, इसे इस्तेमाल करने वाला सावधान रहे"।

Simple Example:

```
public class ThrowsDemo {

    static void riskyMethod() throws InterruptedException {
        Thread.sleep(1000); // 1 सेकंड का इंतज़ार
        System.out.println("Waited for 1 second.");
    }

    public static void main(String[] args) {
        try {
            riskyMethod();
        } catch (InterruptedException e) {
            System.out.println("Interrupted!");
        }
    }
}
```

Quick Comparison Table for Aakash Sen's Notes

Keyword	Type	Purpose (उद्देश्य)
finally	Block	रिसोर्स क्लोज करना (जैसे फाइल या डेटाबेस)।
throw	Statement	एक नया एक्सेप्शन ऑब्जेक्ट "फेंकना"।
throws	Signature	आने वाले एक्सेप्शन की घोषणा (Declare) करना।

- **Throw** = "बम फेंकना" (Action)
- **Throws** = "बम की चेतावनी देना" (Warning)
- **Finally** = "धमाके के बाद सफाई करना" (Cleanup)